Object Oriented Programming (CSEG1044)

Unit 03 – Lecture 03: Multithreading + Synchronization

Tofik Ali

School of Computer Science, UPES Dehradun

February 27, 2026

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Learning Outcomes

- Create threads using `Thread` and `Runnable`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Learning Outcomes

- Create threads using `Thread` and `Runnable`.
- Describe thread lifecycle and understand priority limitations.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Learning Outcomes

- Create threads using `Thread` and `Runnable`.
- Describe thread lifecycle and understand priority limitations.
- Identify race conditions and fix them with `synchronized`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Unit 03: Nested Classes, Exceptions, Multithreading & IO Streams

- Lecture focus: Multithreading + Synchronization

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Today: Roadmap

- Concurrency motivation; ‘Thread‘ class and ‘Runnable‘ interface

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Today: Roadmap

- Concurrency motivation; ‘Thread‘ class and ‘Runnable‘ interface
- Thread lifecycle and priorities (what they mean / limitations)

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Today: Roadmap

- Concurrency motivation; ‘Thread‘ class and ‘Runnable‘ interface
- Thread lifecycle and priorities (what they mean / limitations)
- Race conditions and synchronization (‘synchronized‘, basic patterns)

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Concurrency vs Parallelism

- **Concurrency**: tasks overlap in time (good for responsiveness).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Concurrency vs Parallelism

- **Concurrency**: tasks overlap in time (good for responsiveness).
- **Parallelism**: tasks run simultaneously on multiple cores (good for speed).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Creating threads (two ways)

- Extend `Thread` and override `run()`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Creating threads (two ways)

- Extend `Thread` and override `run()`.
- Implement `Runnable` and pass it to `new Thread(r)`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Creating threads (two ways)

- Extend `Thread` and override `run()`.
- Implement `Runnable` and pass it to `new Thread(r)`.
- **Important:** call `start()` to create a new thread.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

start() vs run() (very important)

```
Runnable task = new Runnable() {
    public void run() {
        System.out.println(Thread.currentThread().getName()
            );
    }
};

Thread t = new Thread(task, "worker");

t.run(); // runs on current thread (main)
t.start(); // runs on a new thread (worker)
```

- run() is a normal method call (no new thread).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

start() vs run() (very important)

```
Runnable task = new Runnable() {
    public void run() {
        System.out.println(Thread.currentThread().getName()
            );
    }
};

Thread t = new Thread(task, "worker");

t.run(); // runs on current thread (main)
t.start(); // runs on a new thread (worker)
```

- run() is a normal method call (no new thread).
- start() asks JVM/OS to schedule a new thread.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Waiting for threads: join()

```
Runnable task = new Runnable() {
    public void run() { /* do work */ }
};

Thread t1 = new Thread(task);
Thread t2 = new Thread(task);
t1.start(); t2.start();

t1.join(); // wait for t1 to finish
t2.join(); // wait for t2 to finish
System.out.println("Both done");
```

- Without `join()`, main may finish early.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Waiting for threads: join()

```
Runnable task = new Runnable() {
    public void run() { /* do work */ }
};

Thread t1 = new Thread(task);
Thread t2 = new Thread(task);
t1.start(); t2.start();

t1.join(); // wait for t1 to finish
t2.join(); // wait for t2 to finish
System.out.println("Both done");
```

- Without `join()`, main may finish early.
- `join()` throws `InterruptedException`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Lifecycle + priorities (practical)

- New → Runnable/Running → Waiting/Blocked → Terminated

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Lifecycle + priorities (practical)

- New → Runnable/Running → Waiting/Blocked → Terminated
- Waiting happens due to sleep, locks, I/O, etc.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Lifecycle + priorities (practical)

- New → Runnable/Running → Waiting/Blocked → Terminated
- Waiting happens due to sleep, locks, I/O, etc.
- Priority is a **hint**, not a guarantee (depends on OS/JVM).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Race condition (core problem)

- Multiple threads modify shared state at the same time.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Race condition (core problem)

- Multiple threads modify shared state at the same time.
- Result depends on timing/interleaving.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Race condition (core problem)

- Multiple threads modify shared state at the same time.
- Result depends on timing/interleaving.
- Example: `value++` is not atomic (read-modify-write).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Synchronization with synchronized

- Protect a critical section using a lock/monitor.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Synchronization with `synchronized`

- Protect a critical section using a lock/monitor.
- Only one thread at a time can execute synchronized code on the same lock.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Synchronization with `synchronized`

- Protect a critical section using a lock/monitor.
- Only one thread at a time can execute synchronized code on the same lock.
- Keep synchronized sections small; avoid deadlocks.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Example: synchronized counter

```
class SafeCounter {
    private int value = 0;
    synchronized void inc() { value++; }
    synchronized int get() { return value; }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Thread demo (two threads) (1/2)

```
public class Demo {
    public static void main(String[] args) throws
        InterruptedException {
        SafeCounter c = new SafeCounter();

        Runnable task = new Runnable() {
            public void run() {
                for (int i = 0; i < 100000; i++) c.inc();
            }
        };
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Thread demo (two threads) (2/2)

```
// continued...
        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start();
        t2.start();
        t1.join(); t2.join();
        System.out.println(c.get()); // expected 200000
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Mini Exercises (2–3 mins)

1. Difference: `start()` vs `run()`?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Mini Exercises (2–3 mins)

1. Difference: `start()` vs `run()`?
2. Why is `value++` unsafe in multiple threads?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Mini Exercises (2–3 mins)

1. Difference: `start()` vs `run()`?
2. Why is `value++` unsafe in multiple threads?
3. What does `synchronized` guarantee (in one sentence)?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Key Takeaways

- Threads enable concurrency for responsiveness and throughput.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Key Takeaways

- Threads enable concurrency for responsiveness and throughput.
- Race conditions occur with shared mutable state.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Key Takeaways

- Threads enable concurrency for responsiveness and throughput.
- Race conditions occur with shared mutable state.
- `synchronized` protects critical sections using a lock.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Checkpoint Questions

1. What is a race condition?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Checkpoint Questions

1. What is a race condition?
2. Why is thread priority not a guarantee?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Checkpoint Questions

1. What is a race condition?
2. Why is thread priority not a guarantee?
3. How does `synchronized` help?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

Next Lecture Preview

Next: Java I/O Streams + FileReader/FileWriter

- We will read/write files and handle errors safely with exceptions.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).